

AutoCkt 安裝步驟

本機 Linux 環境設定 (New version)

For MacOS

1. 開啟「終端機 (Terminal)」(位於 應用程式 -> 工具程式)
2. 安裝 Homebrew (已安裝可跳過)

```
$ /bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

前置套件安裝

```
$ brew install gcc libtool automake autoconf flex bison wget git
```

For Win10/11

1. 點擊 Windows 開始選單，搜尋「開啟或關閉 Windows 功能」(Turn Windows features on or off)。
2. 在清單中找到並勾選以下兩項：
 - 「適用於 Linux 的 Windows 子系統」(Windows Subsystem for Linux)
 - 「虛擬機器平台」(Virtual Machine Platform)
3. 按確定後，系統會要求重新啟動電腦。
4. 重新啟動後，在開始選單搜尋「PowerShell」，對其點擊右鍵並選擇「以管理員身分執行」。
5. 在視窗中輸入以下指令：

```
$ wsl --install
```

這條指令會自動幫您下載最新的 Linux 核心並安裝預設的 *Ubuntu* 分發版。

* 如果系統提示已安裝過，但您沒有 Ubuntu，可以輸入：

```
$ wsl --install -d Ubuntu
```

6. 從開始選單搜尋"Ubuntu"開啟，輸入你自己的 username 與 password

7. 執行以下指令

```
$ sudo apt update && sudo apt upgrade -y
```

```
$ sudo apt install build-essential flex bison libx11-dev libxaw7-dev git  
wget -y
```

安裝 Anaconda

1. 下載 Anaconda Linux 版本

```
$ wget https://repo.anaconda.com/archive/Anaconda3-2025.06-0-Linux-  
x86_64.sh
```

2. 執行安裝腳本

```
$ bash Anaconda3-2025.06-0-Linux-x86_64.sh
```

3. 遵循安裝步驟

```
[Enter] >> “yes” >> [Enter] >> “no”
```

```
[q313510175@tsri-202-201 ~]$ bash Anaconda3-2025.06-0-Linux-x86_64.sh  
Welcome to Anaconda3 2025.06-0  
  
In order to continue the installation process, please review the license  
agreement.  
Please, press ENTER to continue  
>>>
```

```
By continuing installation, you hereby consent to the Anaconda Terms of Service available at https://anaconda.com/legal.  
  
Do you accept the license terms? [yes|no]  
>>> yes
```

```
Anaconda3 will now be installed into this location:  
/mseda_nas/msedalab/q313510175/anaconda3  
  
- Press ENTER to confirm the location  
- Press CTRL-C to abort the installation  
- Or specify a different location below  
  
[/mseda_nas/msedalab/q313510175/anaconda3] >>>
```

```

Downloading and Extracting Packages:
Preparing transaction: done
Executing transaction: done
entry_point.py:256: DeprecationWarning: Python 3.14 will, by default, filter extracted tar archives and reject files or modify their metadata. Use the filter argument to control this behavior.
installation finished.
Do you wish to update your shell profile to automatically initialize conda?
This will activate conda on startup and change the command prompt when activated.
If you'd prefer that conda's base environment not be activated on startup,
run the following command when conda is activated:

conda config --set auto_activate_base false

You can undo this by running `conda init --reverse $SHELL`? [yes|no]
[no] >>> no

```

4. 環境設定

- 確認安裝成功：\$ ~/anaconda3/bin/conda --version
- 初始化：\$ ~/anaconda3/bin/conda init bash

```
$ source ~/.bashrc
```

5. 設定完，測試 conda 版本

```
$ conda --version
```

安裝 NGSpice

1. 回到家目錄

```
$ cd ~
```

2. 下載 ngspice-27.tar.gz

```
$ wget https://sourceforge.net/projects/ngspice/files/ng-spice-rework/old-releases/27/ngspice-27.tar.gz
```

3. 解壓縮

```
$ tar -zxvf ngspice-27.tar.gz
```

4. 進到 ngspice 資料夾，設定 configuration

```
$ cd ngspice-27
```

```
$ ./configure --enable-xspice --disable-debug --with-readline=no --with-x=no --prefix=$HOME/ngspice-27/opt
```

- --with-x=no: 不使用 ngspice 內建的畫圖環境 (root 可能沒安裝那個環境)
- --prefix=\$HOME/ngspice-27/opt: 改寫 ngspice 的安裝路徑到 local 資料夾

5. makefile 安裝 ngspice

```
$ make -j`nproc`
```

```
$ make install
```

- `make -j`nproc``: `-j` 指多核心平行編譯，``nproc`` 指系統核心數

6. 確認安裝與設定成功

i. 可執行檔的位置

```
$ ls $HOME/ngspice-27/opt/bin
```

```
[313510175@mseda01 ~]$ ls $HOME/opt/ngspice-27/bin  
cmpp ngmakeidx ngmultidec ngnutmeg ngproc2mod ngsconvert ngspice  
[313510175@mseda01 ~]$
```

ii. 設定環境變數，並重新載入

```
$ export PATH=$HOME/ngspice-27/opt/bin:$PATH
```

iii. 設定完，查看 ngspice 版本

```
$ ngspice -v
```

```
[313510175@mseda01 ~]$ ngspice -v  
ngspice compiled from ngspice revision 27  
Written originally by Berkeley University  
Currently maintained by the NGSpice Project  
  
Copyright (C) 1985-1996, The Regents of the University of California  
Copyright (C) 1999-2011, The NGSpice Project  
[313510175@mseda01 ~]$
```

安裝 AutoCkt

1. 回到家目錄

```
$ cd ~
```

2. 下載 AutoCkt

```
$ git clone https://github.com/ksettaluri6/AutoCkt.git
```

3. 進到 AutoCkt 資料夾，用 conda 安裝需要的套件

```
$ cd AutoCkt
```

```
$ conda env create -f environment.yml
```

4. 啟動環境

```
$ conda activate autockt
```

5. 改動部分程式碼

在資料夾中找到下列幾個檔案，並修改其中內容

- /eval_engines/ngspice/ngspice_wrapper.py

修改 line 18 如下圖所示

```
16 class NgSpiceWrapper(object):
17
18     BASE_TMP_DIR = os.path.expanduser("~/AutoCkt/ckt_da")
```

ii. /autockt/val_autobag_ray.py

改動 ray.init()

```
1 import ray
2 import ray.tune as tune
3 from ray.rllib.agents import ppo
4 from autockt.envs.ngspice_vanilla_opamp import TwoStageAmp
5
6 import argparse
7 parser = argparse.ArgumentParser()
8 parser.add_argument('--checkpoint_dir', '-cpd', type=str)
9 args = parser.parse_args()
10
11 import os
12 tmp = os.environ.get("RAY_TMPDIR")
13 #ray.init()
14 ray.init(temp_dir=tmp)
```

iii. /autockt/rollout.py

用我們給的 rollout.py 覆蓋掉原本檔案的 code

6. 訓練 AutoCKT

i. 連接 library files

```
$ python eval_engines/ngspice/ngspice_inputs/correct_inputs.py
```

ii. 生成 spec

```
$ python autockt/gen_specs.py --num_specs ###
```

- spec 檔為 pickle 檔，會生在 autockt/gen_specs/裡面
- ### 為生成的 spec 組數，如果要再現 github 中的 results，要設定

350

iii. 設定環境變數

```
$ export PATH=$HOME/ngspice-
```

```
27/opt/bin:/$PATH
```

```
$ export PYTHONPATH=$PWD
```

```
$ mkdir -p ray_tmp
```

```
$ export
```

```
RAY_TMPDIR=$HOME/AutoCkt/ray_tmp
```

iv. 開啟 ipython 介面

```
$ ipython
```

```
Result for PPO_TwoStageAmp_0:
custom_metrics: {}
date: 2025-09-22_15-51-32
done: false
episode_len mean: 30.0
episode_reward_max: -19.707433740008813
episode_reward_mean: -28.71682060453251
episode_reward_min: -41.82980005111106
episodes_this_iter: 36
episodes_total: 36
experiment_id: e17c81f302164ffe81a4fa191f066d3f
hostname: mseda01
info:
  default:
    cur_kl_coeff: 0.20000000298023224
    cur_lr: 4.999999873689376e-05
    entropy: 7.681983947753906
    kl: 0.008173730224370956
    policy_loss: -0.0456894151866436
    total_loss: 228.11032104492188
    vf_explained_var: 0.01809680461883545
    vf_loss: 228.1543731689453
  grad_time_ms: 1442.948
  load_time_ms: 62.7
  num_steps_sampled: 1200
  num_steps_trained: 1200
  sample_time_ms: 22471.213
  update_time_ms: 1004.894
  iterations_since_restore: 1
  node_ip: 140.113.201.195
  num_metric_batches_dropped: 0
  pid: 5595
  policy_reward_mean: {}
  time_since_restore: 25.04340386390686
  time_this_iter_s: 25.04340386390686
```

v. 開始訓練

```
%run autockt/val_autobag_ray.py
```

- 若開始正常跑，應該過不到一分鐘會出現右上圖的訊息
- 整個訓練過程根據設定的 num_specs 越大，需要的時間越久
- RL 訓練過程中每個 iteration 產生的電路在
~/AutoCkt/ckt_da/designs_two_stage_opamp/裡面
- RL 訓練過程中的 checkpoint 會在
~/ray_results/train_45nm_ngspice/PPO_opamp-v0_xxx/裡面

7. 驗證 AutoCKT

根據上面訓練出來的 RL agent 來調參數，餵進多組新的 spec，來看看有多少組能達成 spec 的要求

- i. control D 跳出 ipython 介面，輸入以下指令將生成新的驗證 spec 組數，看你想要餵多少組 spec 來進行驗證：

```
$ python autockt/gen_specs.py --num_specs ###
```

- ii. 進入 ipython，輸入驗證指令

```
%run autockt/rollout.py /abs/path/to/checkpoint --run PPO --env  
opamp-v0 --num_val_specs ### --traj_len ## --no-render
```

- /abs/path/to/checkpoint :

RL 訓練的過程中會產出 checkpoint，它代表在 training 中某個 episode 下訓練出的 agent，會在

~/ray_results/train_45nm_ngspice/PPO_opamp-v0_xxx/checkpoint_%%%/checkpoint-%%%裡面。

xxx 會是執行的時間點，%%%為 checkpoint 編號。可以挑一個 checkpoint 作為驗證的 model，接著用絕對路徑來替換掉

/abs/path/to/checkpoint

- --num_val_specs ### :

替換成新生成的驗證 spec 組數，例如 --num_val_specs 50 代表從規格空間隨機取 50 組來測試模型，看看 agent 能否設計出滿足這些規格的電路。

- --traj_len ## :

每一組規格允許 agent 嘗試的最大步數 (trajectory 長度)，例如 --traj_len 30: agent 最多可以在 30 步內調參數 → 模擬 → 收到回饋，直到滿足規格或超時。

iii. 最後可能會顯示像這樣的訊息

- Episode reward → 越靠近 1 越好
- Specs reached: 44/6 → 44 組有達標，6 組沒有達標
- Num specs reached: 44/50 → 50 組驗證規格中有 44 組達標

```
-----
params = [17 14 87 13 5 25 68]
specs: [3.83609010e+02 2.46211100e-04 6.05622524e+01 1.13321670e+07]
ideal specs: [3.99000000e+02 1.04731608e-03 6.00000000e+01 6.83641821e+06]
re: 10
-----
[array([1]), array([0]), array([2]), array([0]), array([0]), array([0]), array([2])]
10
True
Episode reward 0.8766924501795419
Specs reached: 44/6
Num specs reached: 44/50
In [2]:
```

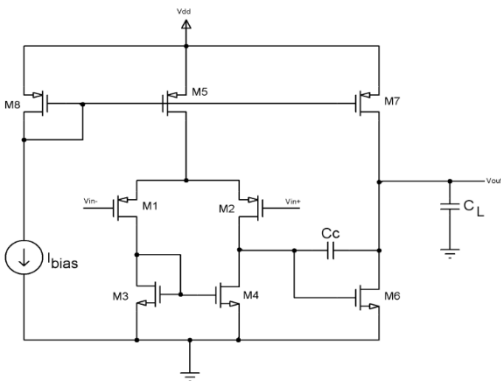
細節與補充

1. AutoCkt 使用的製程: 45nm BSIM predictive technology

- i. **45nm BSIM 預測技術**是用於預測 45 奈米技術節點下 MOSFET 行為和性能的建模技術與製程參數。
- ii. BSIM (柏克萊短通道 IGFET 模型) 是描述 MOSFET 電氣行為的廣泛使用模型。45 奈米節點是半導體縮放的重要步驟，需要準確的模型來預測設備性能，包括閾值電壓 (V_{th})、跨導、亞閾值斜率以及各種條件下的漏電流。
- iii. 此 predictive technology 已包含在 github 中，**不用再額外安裝**。位置：
AutoCkt/eval_engines/ngspice/ngspice_inputs/spice_models/45nm_bulk.txt

2. AutoCkt 使用的電路: two stage op-amp

- i. 電路模板位置：
AutoCkt/eval_engines/ngspice/ngspice_inputs/netlist/two_stage_opamp.cir



```
1 two_stage_amp test
2
3 * Two stage OPAMP
4 .include "/mseda_nas/course/2025CAD/2025CAD123/AutoCkt/eval_engines/ngspice/ngspice_inputs/spice_models/45nm_bulk.txt"
5
6 .param wp1=0.5u lp1=90n mp1=10
7 .param wn1=0.5u ln1=90n mn1=38
8 .param wp3=0.5u lp3=90n mp3=9
9 .param wp3=0.5u lp3=90n mp3=4
10 .param wn4=0.5u ln4=90n mn4=20
11 .param wn5=0.5u ln5=90n mn5=60
12 .param cc=3p
13 .param ibias=30u
14 .param cload=10p
15 .param vcm=0.6
16
17 mp1 net4 net4 VDD VDD pmos w=wp1 l=lp1 m=mp1
18 mp2 net5 net4 VDD VDD pmos w=wp1 l=lp1 m=mp1
19 mn1 net4 net2 net3 net3 nmos w=wn1 l=ln1 m=mn1
20 mn2 net5 net1 net3 net3 nmos w=wn1 l=ln1 m=mn1
21 mn3 net3 net7 VSS VSS nmos w=wn3 l=ln3 m=mn3
22 mn4 net7 net7 VSS VSS nmos w=wn4 l=ln4 m=mn4
23 mp3 net6 net5 VDD VDD pmos w=wp3 l=lp3 m=mp3
24 mn5 net6 net7 VSS VSS nmos w=wn5 l=ln5 m=mn5
25 cc net5 net6 cc
26 ibias VDD net7 ibias
27
28 vin in 0 dc=0 ac=1.0
29 ein1 net1 cm in 0 0.5
30 ein2 net2 cm in 0 -0.5
31 vcm cm 0 dc=vcm
32
33 vdd VDD 0 dc=1.2
34 vss 0 VSS dc=0
35 CL net6 0 cload
```